

Supplementary Material

RIsearch and RIOT: An integrated, high-performance framework for RNA–RNA interaction and siRNA off-target prediction

Lorenzo Favaro

Supplementary Methods

RIsearch core search benchmark

RIsearch reimplements the seed-and-extend search of RIsearch2 (Alkan et al., 2017) in Rust. To quantify the resulting speed-up, both tools were benchmarked on a common workload: 63 siRNA guide sequences searched against a chromosome-X target set of Ensembl human cDNA transcripts filtered to ≥ 80 nt, on an AMD EPYC 9554 server. Both tools were supplied with pre-built indexes of the same target and run with identical search parameters, so that only search time is compared and index construction is excluded. Input files were staged on local disk to remove NFS latency, and wall-clock time was recorded with Hyperfine (one warm-up run followed by three timed runs, with the output directory cleared before each run). In all figures the Rust implementation is labelled **RIsearch_RS**.

The parameter grid comprised seed schemes 2:7, 2:7/5, 1:12/6, 1:12/7, 1:12/8 and the singletons 4, 5, 6, 12 and 16; extension lengths 0, 5, 10, 15 and 20 nt; mismatch counts 0–3; and output formats p1–p4. Thread scaling was measured for seed scheme 6 at 1, 2, 4, 8, 16 and 32 threads. The energy threshold was -10 kcal mol $^{-1}$ for extension ≥ 5 nt and 0 for seed-only runs (extension < 5 nt, for which no dynamic-programming extension is performed). The two search commands, shown for seed scheme 2:7/5 with extension 20 nt on 32 threads, were:

```
# RIsearch2 (C)
risearch2.x -q sirnas.fa -i chrX.pksuf -s 2:7/5 \
  -e -10 -l 20 --threads 32 -z t99 -p

# RIsearch (Rust; RIsearch_RS)
risearch search --query sirnas.fa --target chrX.idx --seed 2:7/5 \
  --energy -10 --extension 20 --threads 32 --matrix t99 \
  --no-dedup -p 1 --multifile --compress gzip --output results_dir
```

With matched gzip-compressed output, RIsearch runs about twice as fast as RIsearch2 on the canonical seed-and-extend schemes (Fig. S1; median 1.9 $\times$, up to 2.3 $\times$), the advantage growing with extension length as the Gotoh dynamic-programming stage comes to dominate runtime (Fig. S3) and holding across thread counts (Fig. S2). RIsearch additionally offers zstd-compressed output, which RIsearch2 lacks; substituting zstd (Figs. S4, S5) widens the margin over RIsearch2 to a median of 2.3 $\times$ on seed-and-extend workloads and up to $\sim 3\times$ on seed-only searches.

Index construction

RIsearch builds its suffix-array index in parallel across cores with libsaïs (Grebakov), whereas RIsearch2’s libdivsufsort construction is single-threaded. Building the GRCh38/hg38 index was benchmarked against RIsearch2 with Hyperfine (Fig. S6). When the index is written to

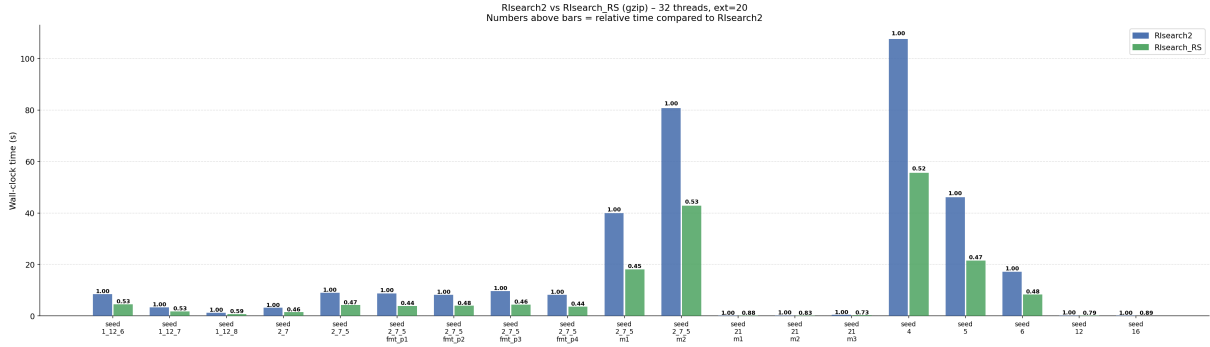


Figure S1: **Per-configuration search time of Rlsearch versus Rlsearch2.** Wall-clock time at 32 threads with extension length 20nt and matched gzip-compressed output, across seed schemes, output formats (fmt_p1–p4) and mismatch counts (m1–m3). Numbers above each bar pair give Rlsearch’s time relative to Rlsearch2 (lower is faster). On the canonical seed-and-extend schemes Rlsearch is roughly twice as fast (relative time 0.44–0.53); the margin narrows for long exact-match singleton seeds (12, 16, 21), whose sub-second runtimes are dominated by fixed start-up rather than by search.

disk, Rlsearch is already $\sim 1.5\times$ faster single-threaded ($0.66\times$ the Rlsearch2 time) and $\sim 2.2\times$ faster at 16 threads ($0.45\times$); when the index is instead kept in memory, the parallel suffix-array construction is up to $\sim 5.6\times$ faster ($0.18\times$ at 16 threads), showing that disk I/O, not computation, dominates the write-to-disk case.

RIOT pipeline stages and partition-function model

The four-stage model of Alkan et al. (2017) is preserved. In the first stage, siRNA–target interaction predictions are generated by calling `rlsearch.index()` to build the suffix-array index of the target and `rlsearch.search()` to locate binding sites, both invoked directly through the Python bindings, returning results as a Polars DataFrame in the same process with no intermediate files on disk. In the second stage, predicted binding sites are intersected with an expression annotation (GTF or BED) to retain sites overlapping expressed features and to assign position-specific abundance values. Intersection is performed in-memory using an NCLS interval index (Aleksyenko and Lee, 2007) partitioned by chromosome and strand, with chromosome–strand pairs distributed across a thread pool; no external tools such as `bedtools` are needed. In the third stage, per-site binding-site opening energies $O_i = -RT \ln p_i$ are either computed on-the-fly from a genome FASTA or read from per-chromosome Parquet profiles pre-computed by the `accessibility` subcommand, in both cases using the RNAplfold algorithm (ViennaRNA 2.7.2 Python API (Lorenz et al., 2011a); $W=80$, $L=40$, $u=30$) at the same temperature used for the partition function, so that E'_i and O_i are on a consistent thermodynamic scale. The Boltzmann-weighted sum

$$Z_s = \sum_{i \in \mathcal{I}_s} A_i \exp\left(\frac{-(E'_i + O_i)}{RT}\right) \quad (1)$$

is evaluated as a single vectorised `group_by` aggregation in Polars, where A_i is the transcript expression level, E'_i is the Rlsearch hybridisation energy, $O_i = -RT \ln p_i$ is the opening energy derived from the RNAplfold unpaired probability p_i , $R = 1.987 \times 10^{-3} \text{ kcal mol}^{-1} \text{ K}^{-1}$ is the molar gas constant, and T is the folding temperature (default 310.15 K, 37°C). When an on-target binding site is supplied via `--on-target-ids`, its Boltzmann weight W_{on} is computed identically and added to Z_s ; the on-target probability $p_{\text{on}} = W_{\text{on}}/Z_s$ determines the aggregate off-targeting potential $P_{\text{OFF}} = 1 - p_{\text{on}}$. Per-transcript off-targeting probabilities $p_{\text{off},i} = W_i/Z_s$

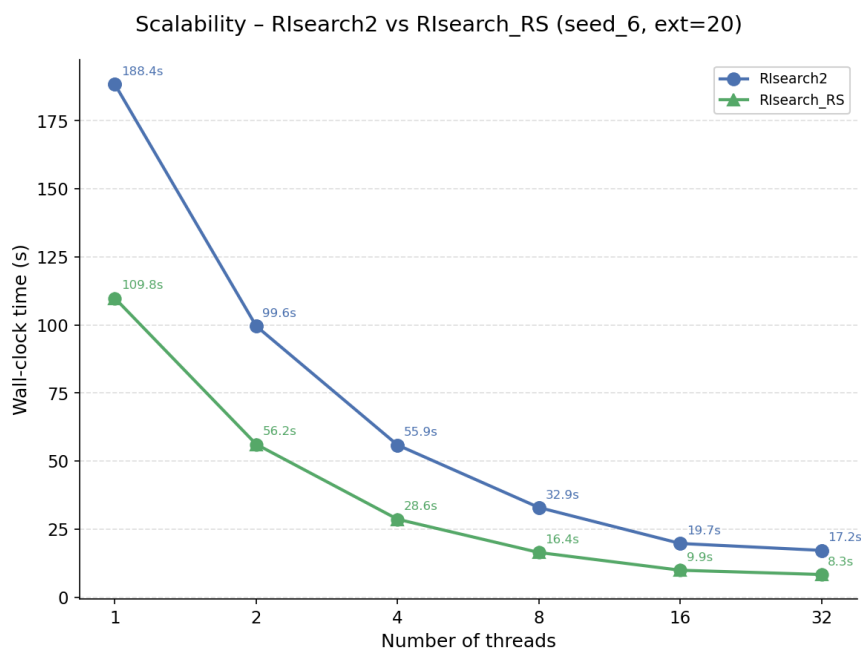


Figure S2: **Thread scaling of RIssearch versus RIssearch2** (seed scheme 6, extension 20 nt). RIssearch is faster at every thread count, from $1.7\times$ single-threaded (RIssearch2 188 s vs. RIssearch 110s) to $2.1\times$ at 32 threads (17.2s vs. 8.3s). Both tools scale near-linearly up to 16 threads.

are then computed as column operations on the resulting DataFrame. Parametric sweeps over α , γ , and θ are resolved in a single aggregation pass rather than repeated sequential iterations. In the fourth stage, results are written incrementally as each siRNA completes, in TSV, CSV, or Parquet format.

Parallelism and memory

When processing a directory of per-siRNA prediction files, RIOT distributes work across a configurable process pool. A spawn-based multiprocessing context is used to avoid interaction with the Rayon thread pool used internally by Polars. The expression annotation is loaded once into Arrow IPC format and memory-mapped into each worker process via the operating-system page cache, keeping peak memory proportional to a single annotation copy regardless of worker count. Each worker performs the intersection, accessibility lookup, and partition-function evaluation for one siRNA at a time; prediction files are parsed lazily so that only filtered, selected columns are materialised, and output is streamed to disk incrementally throughout a batch run.

Benchmark dataset

The benchmark uses a subset of the genome-wide siRNA efficacy library of [Hüsken et al. \(2005\)](#), which comprises 2,431 chemically synthesised 21-mer siRNAs targeting 34 human genes, with experimentally measured knockdown efficacies obtained in the H1299 non-small-cell lung carcinoma cell line. A master set of 2,000 siRNAs was drawn by uniform random sampling (without replacement) from this library; smaller batch sizes of 1,000 and 500 were produced by further subsampling from this master set, with a fresh random draw for each of the five independent benchmark iterations. Target sequences for the off-target search were derived from the GRCh38/hg38 reference genome (UCSC assembly, December 2013). Expression values were taken from an H1299 RNA-seq experiment (ArrayExpress accession E-MTAB-2770), converted to per-transcript read counts and stored in BED format.

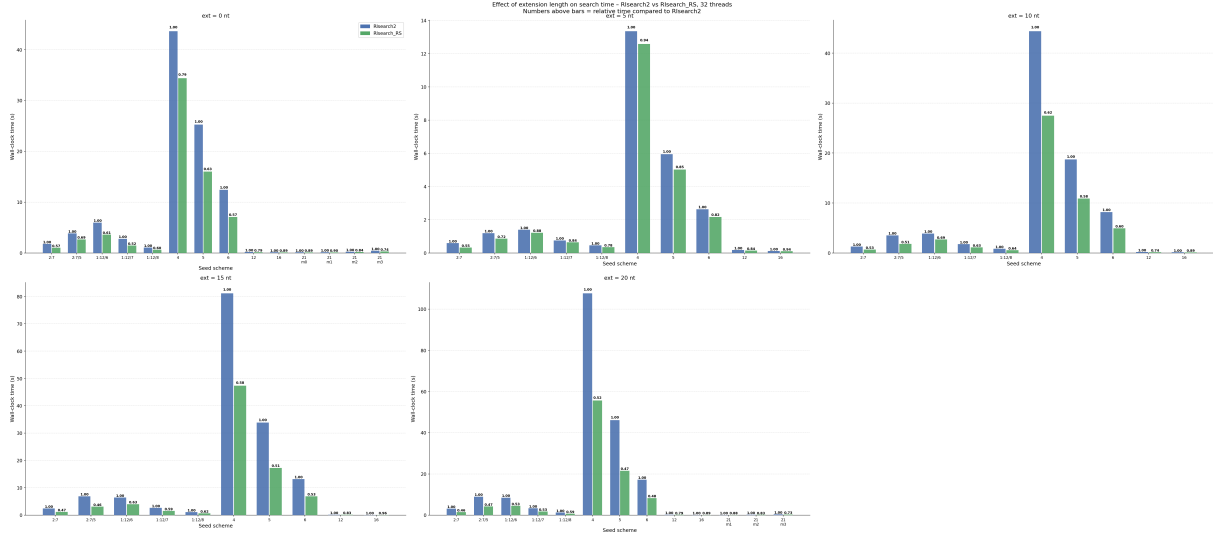


Figure S3: **Effect of extension length on search time** (0–20 nt, 32 threads, matched gzip output). Numbers above bars give RIsearch’s time relative to RIsearch2. RIsearch’s advantage grows with extension length as the Gotoh dynamic-programming stage comes to dominate run-time; at extension 0 (seed matching only) the two tools are closer.

Genome indexing and RIsearch2 binding-site search

The hg38 reference genome was indexed with RIsearch2 to produce a suffix-array package file (hg38.pksuf):

```
risearch2.x -c hg38.fa -o hg38.pksuf
```

where `-c` specifies the target FASTA to index and `-o` names the output suffix-array package. Binding-site predictions for each siRNA were generated once and stored as compressed output files; this search step was excluded from all pipeline benchmark timings so that only the post-search processing stages (intersection, accessibility annotation, partition function, and output) are measured.

The search command used was:

```
risearch2.x \
-i hg38.pksuf \
-q huesken_sirnas.fa \
-s 2:7 \
-e -10 \
-p2 \
-l 25 \
-t 32
```

Key flag meanings:

- `-i hg38.pksuf` — pre-built suffix-array index of the hg38 target.
- `-q huesken_on_subset.fa` — FASTA file of siRNA query sequences (guide strand, 5′→3′).
- `-s 2:7` — restrict seed matches to guide-strand positions 2–7 (the canonical seed region for RISC-mediated target recognition).
- `-e -10` — energy threshold of $-10 \text{ kcal mol}^{-1}$; candidate sites with a hybridisation energy above this value are discarded.

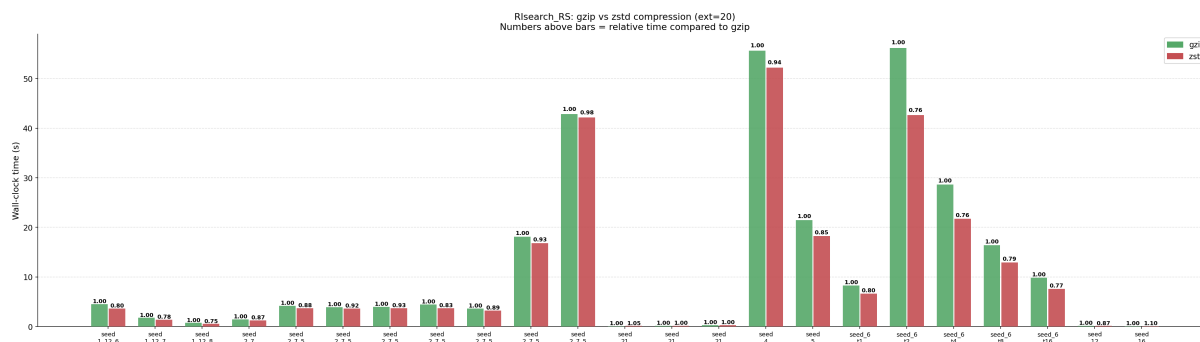


Figure S4: **Output-compression modes within RISearch:** zstd versus gzip (32 threads, extension 20 nt). Numbers above bars give zstd time relative to gzip. zstd trims up to ~25% of wall-clock time (relative time 0.75–0.98) because compression, not search, is accelerated. RISearch2 offers only gzip-style output, so the head-to-head comparisons in Fig. S1 use gzip for both tools; with zstd, RISearch’s advantage over RISearch2 widens further (quantified in the text above).

- `-p2` — bindingsite output format, reporting one line per predicted interaction with genomic coordinates, energies, and alignment.
- `-l 25` — minimum alignment length of 25 nt, filtering out truncated partial matches at sequence boundaries.
- `-t 32` — use 32 parallel threads for the search.
- `--verbose` — write progress information to standard error.

H1299 expression annotation

Per-transcript expression weights for the H1299 (NCI-H1299, large-cell lung carcinoma) cell line were derived from the RNA-seq dataset E-MTAB-2770 (*RNA-Seq of 675 human cancer cell lines*, CCLE), which is archived at ArrayExpress and contains data for approximately 675 cancer cell lines.

Sample identification. The SDRF file was queried to identify the single H1299 paired-end run, yielding accession SRR8615898 (R1: 4.7 GB; R2: 4.8 GB).

Reference genome and annotation. GENCODE v47 was used as the reference (GRCh38 coordinate space): `gencode.v47.transcripts.fa.gz` for Salmon index construction and `gencode.v47.annotation.gtf` for transcript coordinates.

Quantification. A Salmon v1.10.3 quasi-mapping index was built from the GENCODE transcript sequences with the `--gencode` flag (strips pipe-delimited header fields so transcript IDs match the GTF). Paired-end reads were quantified with:

```
salmon quant \
  -i input/salmon_index \
  -l A \
  -1 SRR8615898_1.fastq.gz \
  -2 SRR8615898_2.fastq.gz \
  -p 16 \
  --validateMappings \
```

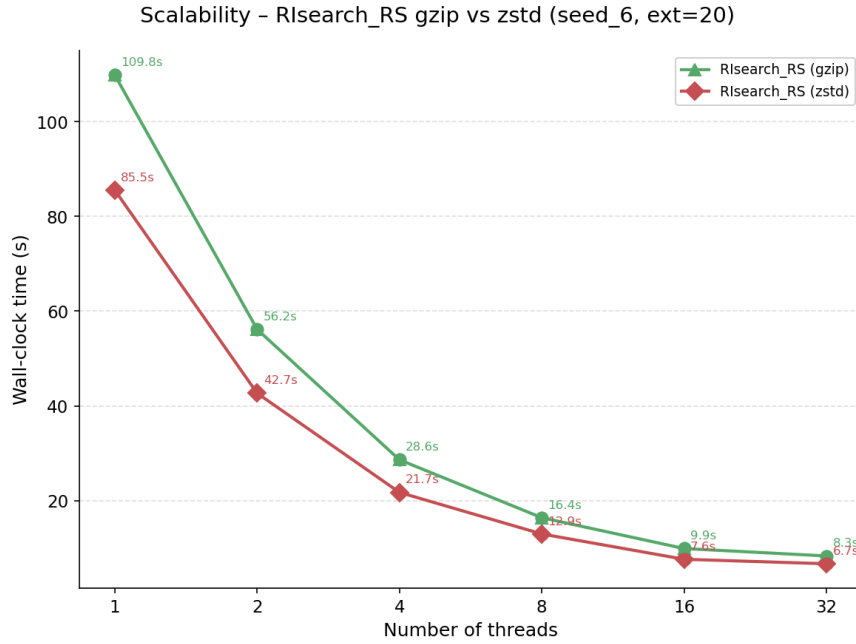


Figure S5: **Thread scaling of Rlsearch output compression** (seed 6, extension 20 nt): zstd is consistently faster than gzip, e.g. 6.7 s vs. 8.3 s at 32 threads.

```
--gcBias \
-o input/quant/SRR8615898
```

Key flags: `-l A` auto-detects library strandedness; `--validateMappings` applies alignment scoring to filter spurious mappings; `--gcBias` corrects for GC-content bias. The run achieved an 87.6% mapping rate.

BED7 construction. Salmon output (`quant.sf`, 384,354 transcript rows) was merged with GTF transcript-level coordinates using a custom script (`make_bed7.py`). Each GTF **transcript** record was converted to 0-based BED coordinates (BED start = GTF start -1 ; end unchanged) and joined to its TPM value by versioned ENST identifier. Transcripts absent from the Salmon output (PAR-locus duplicates and assembly patches) received TPM = 0. The output file (`annotation_E-MTAB-2770_GRCh38_v47.bed`) is a tab-separated BED7 file (385,659 rows; no header) in which column 7 carries the per-transcript TPM as a floating-point value:

chrom	start	end	transcript_id	.	strand	TPM
chr12	6534516	6538371	ENST00000229239.10	.	+	6130.06

Of the 385,659 transcripts, 143,881 (37.3%) carried TPM > 0, consistent with the fraction of GENCODE transcripts detectably expressed in a single cell-line condition. Transcripts with TPM = 0 are retained in the file because the intersection service reads the annotation as a single indexed structure; dropping unexpressed entries would misalign row offsets. The high GAPDH TPM (ENST00000229239.10: 6,130) confirms biologically plausible quantification.

Benchmark protocol

Timing and peak-memory measurements were collected on a server equipped with an AMD EPYC 9554 processor (64 physical cores, 128 threads) and sufficient RAM to keep all input files resident. Both pipelines used 32 parallel workers (`-j 32` for GNU parallel in the legacy case; `-j 32` process-pool workers for the new pipeline).

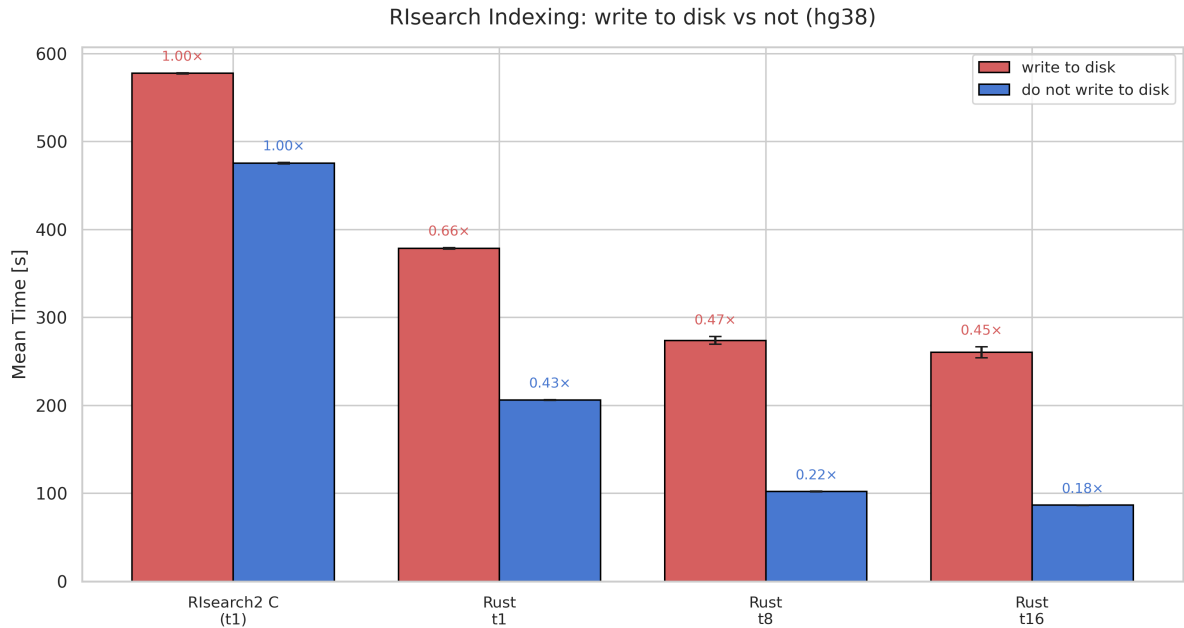


Figure S6: **Suffix-array index construction time for the GRCh38/hg38 genome:** Rlsearch2 (C, single-threaded) versus Rlsearch (Rust) at 1, 8 and 16 threads, each with and without writing the index to disk. Numbers above bars give the time relative to the corresponding Rlsearch2 baseline (lower is faster). Writing to disk, Rlsearch builds the index $\sim 2.2\times$ faster at 16 threads ($0.45\times$); kept in memory, its parallel construction is up to $\sim 5.6\times$ faster ($0.18\times$), indicating that disk I/O dominates the write-to-disk timing.

For each combination of pipeline version (new / legacy) and batch size (500, 1,000, 2,000 siRNAs), the following procedure was applied: (i) one warmup run was executed and discarded to populate OS page-cache entries and JIT any Python bytecode; (ii) five timed runs were executed, each with a fresh output directory to avoid I/O caching effects. Wall-clock time and peak resident-set size (RSS) of the parent process were recorded with `/usr/bin/time -v` (GNU time). Because both pipelines spawn independent child processes (GNU parallel in the legacy case; a spawn-based process pool in RIOT), the RSS reported reflects only the parent; aggregate memory across all 32 workers will exceed this figure.

For batch sizes of 1,000 and 500, each of the five timed runs used an independently drawn random subset of the master 2,000-siRNA set, ensuring that variance estimates reflect genuine sampling variability rather than run-to-run measurement noise alone. Pre-computed Rlsearch2 binding-site prediction files were provided as input to both pipelines, so that search time is excluded from all measurements.

Accessibility pre-computation

The legacy pipeline computed per-nucleotide RNA opening energies with a Python 2.7 script that wrapped the `RNAplfold` executable (ViennaRNA 2.6.3) as a subprocess, emitting two flat `uint8` binary files per chromosome (one per strand). RIOT's `accessibility` subcommand reproduces the same folding parameters ($W=80$, $L=40$, $u=30$) through the ViennaRNA 2.7.2 Python API directly, without spawning subprocesses: for each chromosome the call `RNA.fold_compound(...).probs_window` returns per-position unpaired-probability vectors at the specified temperature (default 37°C , set via `--temperature`), from which opening energies $O_i = -RT \ln p_i$ are computed in-process and written as a single per-chromosome Apache Parquet file with columnar compression and zero-copy Arrow reads.

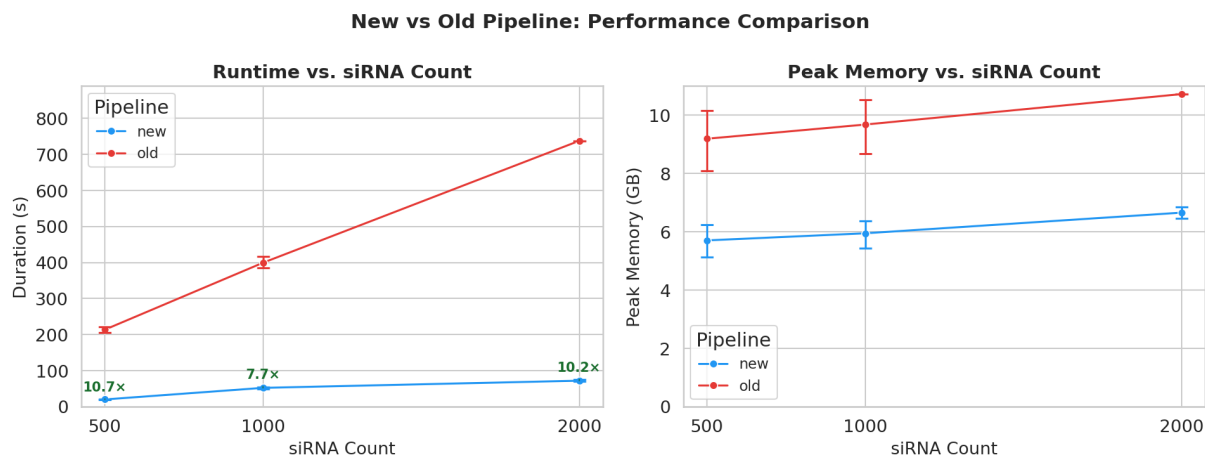


Figure S7: **Wall-clock time and peak memory of RIOT versus the legacy pipeline across batch sizes.** Each box spans the interquartile range of five independent randomised trials; the central line marks the median; whiskers extend to the full range. **(A)** Wall-clock time (seconds) for the intersection, accessibility annotation, Boltzmann partition-function, and output stages on 500, 1,000, and 2,000 siRNAs from the Hüsken et al. genome-wide efficacy library (Hüsken et al., 2005). RIOT achieves a 7.7–10.7 $\times$ speed-up relative to the legacy Python 2.7 implementation. **(B)** Peak resident-set size (GB) of the parent process as reported by `/usr/bin/time -v`. RIOT plateaus at ≈ 7 GB regardless of batch size; the legacy pipeline grows from ~ 8.5 GB to ~ 10 GB as batch size increases. Benchmarks were run on an AMD EPYC 9554 server (64 physical cores) with 32 workers for both pipelines; pre-computed RIssearch2 binding-site predictions were provided as input so that search time is excluded from both measurements.

Accessibility pre-computation benchmark. Folding the complete GRCh38/hg38 reference ($W=80$, $L=40$, $u=30$, 64 workers on the AMD EPYC 9554 server) was compared between the legacy script (driven by GNU `parallel`) and RIOT’s `accessibility` subcommand (Fig. S8, Table S1). RIOT completes the full-genome fold $\sim 34\%$ faster (8 h 46 m vs. 13 h 12 m) and stores the resulting profiles in $\sim 42\%$ less disk space (111.8 GB vs. 192.6 GB), the latter because each chromosome is written as a single two-strand compressed Parquet file rather than two uncompressed binary files. The trade-off is higher peak resident memory (~ 3.6 GB vs. ~ 309 MB, roughly 12 $\times$), as the ViennaRNA Python API holds per-chromosome probability matrices in memory rather than streaming them through a subprocess; both figures are small relative to typical server memory, and this cost is incurred only once per genome and folding condition.

Table S1: Full-genome accessibility pre-computation: legacy script versus RIOT’s `accessibility` subcommand (hg38; $W=80$, $L=40$, $u=30$; 64 workers).

Metric	Legacy	New
Wall-clock time	13 h 12 m	8 h 46 m
Peak resident memory	~ 309 MB	~ 3.6 GB
On-disk profile size	192.6 GB	111.8 GB
Files per chromosome	2	1

S1 Temperature dependent energy parameters

Notation, N indicates an unknown nucleotide. In the stacking energies, the N is substituted with a nucleotide (ACGT/U) giving the stack the highest possible energy. `_` indicates gap, a missing nucleotide, at the corresponding position, that is a bulge on the strand opposite the gap.

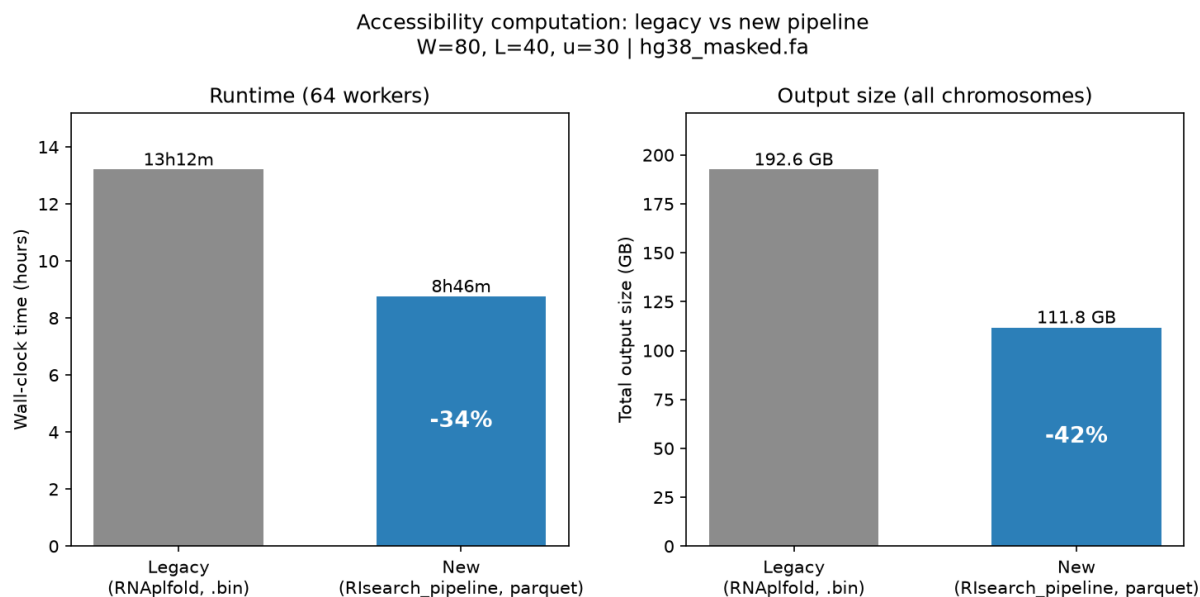


Figure S8: **Full-genome accessibility pre-computation: legacy script versus RIOT's accessibility subcommand** (hg38, $W=80$, $L=40$, $u=30$, 64 workers). **(A)** Wall-clock time: RIOT folds the whole genome in 8 h 46 m versus 13 h 12 m for the legacy RNAplfold script (−34%). **(B)** Total on-disk profile size across all chromosomes: 111.8 GB versus 192.6 GB (−42%), because each chromosome is stored as a single compressed two-strand Parquet file rather than two uncompressed binary files. Peak memory (not shown) is given in Table S1.

RNA: rA,rC,rG,rU,rN,r\
DNA: dA,dC,dG,dT,dN,d\

RNA/RNA: 5'-AC-3' bound to 3'-UG-5': rACUG
DNA/DNA: 5'-AC-3' bound to 3'-TG-5': dACTG
RNA/DNA: 5'-rAC-3' bound to 3'-dTG-5': rACdUG

The energy model of Rlsearch is the sum of the energies of nucleotide stacks, where a stack is two nucleotides bound (aligned) to two nucleotides on the opposite strand or aligned to gaps on the opposite strand when the alignment contains bulges / gaps. A particular case is the stacks initiating or terminating the binding, which are written as, e.g. `_G_C` and `G_C_`, respectively. These stacks are not allowed in other places in the alignment.

The binding energy matrices are derived from those in ViennaRNA [Lorenz et al. \(2011b\)](#), which are again derived from the nearest neighbor energy model parameters from papers [Walter et al. \(1994\)](#); [Turner et al. \(1988\)](#); [Peritz et al. \(1991\)](#); [Jaeger et al. \(1989\)](#); [He et al. \(1991\)](#); [Turner and Mathews \(2010\)](#); [Mathews et al. \(1999, 2004\)](#) cited by ViennaRNA.

RNA/RNA: 2.6.3 RNAduplex

DNA/DNA: 2.6.3 RNAduplex -P DNA

RNA/DNA: 2.1.9h RNAduplex (in source code and binary only)

The RNA/RNA and DNA/DNA matrices are implemented in all later versions of ViennaRNA. However, the RNA/DNA matrices are only found in version 2.1.9h, where they are the default in certain programs including RNAduplex. The first (target) sequence must be the RNA, and the second the DNA, and these should be entered with the corresponding alphabets, that is with U or T, respectively.

When we allow GAPS along with the standard nucleotides, we have $5^4 = 625$ stacking energy parameters (SEP) we need to obtain values for in each of the four cases. However, not all SEPs are allowed, e.g. an alignment must start and end with a binding pair ($_G_C$ is defined, but not $_G_A$) and a gap on both strands is not allowed. In addition, some SEPs are identical due to the nature of the underlying chemistry (rGCAU is the same as rUACG, but rGCdAT is not the same as rUAdCG) and some SEPs are linearly dependent due to the nature of the model (a gap opened must also be closed (an example gap opening could be rAAU $_$ and a closing could be rAA $_U$)).

The undefined SEPs are set to a large positive energy (+20.0 kcal/mol) in the energy matrices, and we enforce linear dependencies according to the rules below

1. A stack of nucleotides is, e.g. rACGU or rACdGT is written as $(ijkl)$
2. Nucleotide are considered pairing when one of AU, AT, CG, GU and GT
3. If i is paired but j is not paired: $(ijkl) = (jilk)$, e.g. rACdGG = rCAdGG
4. For RNA/RNA and DNA/DNA $(ijkl) = (lkji)$
5. For unknown nucleotides (N) the maximum (worst) value is chosen amongst the possibilities for N. In this way we determine the $6^4 = 1296$ SEPs needed for RIsearch.

These rules reduced the number of parameters to be determined from 625 to 266 for RNA/RNA and DNA/DNA and to 459 in RNA/DNA and removed the linear interdependency of the parameters. The actual number of SEPs to be determined is a little lower when illegal stacking values are removed.

The SEPs were obtained by fitting energies and alignments from the respective RNA, DNA and hybrid modes of the RNAduplex according to the list above. To this end we constructed 4 times 50,000 alignments of the following types

1. Exact complimentary sequences
2. GU internal size 1-4 and, terminal, size -4
3. internal loops of size 1-4
4. internal bulges of size 1-4

In each case the sequences were otherwise random and constructed to be 10-20nt complimentary + variation 4nt + 10-20nt complimentary. The alignments obtained from RNAduplex were then parameterized according to the corresponding stacking base pairs and the SEPs were obtained by a linear regression fit of the RNAduplex energy. In cases where the alignments were not full length, the alignments and energies were discarded before fitting.

The RNA/DNA parameters in both the previous versions of RIsearch and in the 2.1.9h version of ViennaRNA were constructed from the Sugimoto 95 paper [Sugimoto et al. \(1995\)](#), which only contains the SEPs for canonical basepair stacks like rACdTG. Missing values were obtained as the average of corresponding RNA/RNA and DNA/DNA SEP. However the RNA/DNA parameters have been improved upon in two papers [Sugimoto et al. \(2000\)](#); [Watkins et al. \(2011\)](#), which respectively adds temperature dependent SEPs for mismatches with neighbouring CG pairs, e.g. rGCGdCAC and for symmetrical mismatches, e.g. rAGdTG and rGGdCG. These values have been used in place of the values obtained by fitting in the final RNA/DNA SEP matrix.

References

- A. V. Alekseyenko and C. J. Lee. Nested Containment List (NCList): a new algorithm for accelerating interval query of genome alignment and interval databases. *Bioinformatics*, 23(11):1386–1393, 2007. doi: 10.1093/bioinformatics/btl647.
- F. Alkan, A. Wenzel, O. Palasca, P. Kerpedjiev, A. F. Rudebeck, P. F. Stadler, I. L. Hofacker, and J. Gorodkin. RIssearch2: suffix array-based large-scale prediction of RNA–RNA interactions and siRNA off-targets. *Nucleic Acids Research*, 45(8):e60, 2017. doi: 10.1093/nar/gkw1325.
- I. Grebnov. libsais: linear-time suffix array, LCP array and Burrows–Wheeler transform construction. <https://github.com/IlyaGrebnov/libsais>. Accessed 2026.
- L. He, R. Kierzek, J. SantaLucia, Jr., A. E. Walter, and D. H. Turner. Nearest-neighbor parameters for gu mismatches: 5’gu3’/3’ug5’ is destabilizing in several contexts but stabilizing in gguc/cugg. *Biochemistry*, 30:11124–11132, 1991.
- D. Hüsken, J. Teisan, M. Lichtner, C. Blochinger, T. Naujoks, W. Paus, B. Schärfke, E. Schoone, A. Bauernfeind, E. Gimenez-Adan, et al. Design of a genome-wide siRNA library using an artificial neural network. *Nature Biotechnology*, 23(8):995–1001, 2005. doi: 10.1038/nbt1118.
- J. A. Jaeger, D. H. Turner, and M. Zuker. Improved predictions of secondary structures for rna. *Proceedings of the National Academy of Sciences USA*, 86:7706–7710, 1989.
- R. Lorenz, S. H. Bernhart, C. Höner zu Siederdisen, H. Tafer, C. Flamm, P. F. Stadler, and I. L. Hofacker. ViennaRNA package 2.0. *Algorithms for Molecular Biology*, 6(1):26, 2011a. doi: 10.1186/1748-7188-6-26.
- R. Lorenz et al. Viennarna package 2.0. *Algorithms for Molecular Biology*, 6:26, 2011b.
- D. H. Mathews, J. Sabina, M. Zuker, and D. H. Turner. Expanded sequence dependence of thermodynamic parameters improves prediction of rna secondary structure. *Journal of Molecular Biology*, 288:911–940, 1999.
- D. H. Mathews et al. Incorporating chemical modification constraints into a dynamic programming algorithm for prediction of rna secondary structure. *Proceedings of the National Academy of Sciences USA*, 101:7287–7292, 2004.
- A. E. Peritz, R. Kierzek, N. Sugimoto, and D. H. Turner. Thermodynamic study of internal loops in oligoribonucleotides: symmetric loops are more stable than asymmetric loops. *Biochemistry*, 30:6428–6436, 1991.
- N. Sugimoto, M. Nakano, and S. Nakano. Thermodynamics–structure relationship of single mismatches in rna/dna duplexes. *Biochemistry*, 39:11270–11281, 2000.
- N. Sugimoto et al. Thermodynamic parameters to predict stability of rna/dna hybrid duplexes. *Biochemistry*, 34:11211–11216, 1995.
- D. H. Turner and D. H. Mathews. Nndb: the nearest neighbor parameter database for predicting stability of nucleic acid secondary structure. *Nucleic Acids Research*, 38:D280–D282, 2010.
- D. H. Turner, N. Sugimoto, and S. M. Freier. Rna structure prediction. *Annual Review of Biophysics and Biophysical Chemistry*, 17:167–192, 1988.
- A. E. Walter et al. Coaxial stacking of helices enhances binding of oligoribonucleotides and improves predictions of rna folding. *Proceedings of the National Academy of Sciences USA*, 91:9218–9222, 1994.

N. E. Watkins et al. Thermodynamic contributions of single internal ra · da, rc · dc, rg · dg and ru · dt mismatches in rna/dna duplexes. *Nucleic Acids Research*, 39:1894–1902, 2011.